But What About Rust?

Design Study for Codex-Assisted rmatch Portability

March 3, 2026

Abstract

This paper proposes a practical research program for deriving a Rust-compatible implementation from the current Java-based rmatch codebase using automated or semi-automated generation. The objective is not blind transpilation, but controlled migration with semantic compatibility, measurable performance, and reproducible evidence. We define the scaffolding artifacts required for Codex to execute this migration reliably, connect those requirements to literature on translation validation, formal language semantics, and performance methodology, and provide an experimental plan that can be run with the existing benchmark infrastructure.

Contents

1 Introduction **2**

2 Algorithmic and Systems Context **2**
2.1 Regex engine background . . . 2
2.2 Why Rust is attractive here . . . 2

3 Research Questions and Hypotheses **2**

4 What Scaffolding Codex Needs **3**
4.1 Scaffolding contract . . . 3
4.2 Why these scaffolds are necessary . . . 3

5 Method: Codex-Assisted Migration Protocol **3**
5.1 Phase A: Baseline freeze and observability . . . 4
5.2 Phase B: Rust compatibility shell . . . 4
5.3 Phase C: Module-by-module generation . . . 4
5.4 Phase D: Translation validation loop . . . 4
5.5 Phase E: Performance characterization . . . 4

6 Experimental Design **5**
6.1 Dependent variables . . . 5
6.2 Acceptance criteria . . . 5

7 Threats to Validity **5**

8 Concrete Deliverables **5**

9 Conclusion **6**

1 Introduction

rmatch is currently implemented in Java as a non-backtracking, multi-pattern matcher with NDFA compilation, lazy DFA construction, and parallel partition execution. The immediate question is whether this implementation can be used as scaffolding for a Rust implementation that is API-compatible, semantically aligned (within declared engine boundaries), and performance-competitive.

The broader context is favorable: robust migration and verification ideas exist in compiler correctness and translation validation [13, 10], while Rust-specific formalization and migration studies are maturing [6, 12, 8, 1, 15, 11, 9].

2 Algorithmic and Systems Context

2.1 Regex engine background

The core algorithmic context for this work follows classical automata-based matching [14, 2], and modern high-throughput matcher engineering (for example Hyperscan in native settings) [16]. rmatch specifically targets large pattern sets where scaling behavior differs from low-pattern-count baselines.

2.2 Why Rust is attractive here

Rust offers memory safety and explicit control over allocation and concurrency behavior, which is relevant for high-load matching pipelines. However, these benefits are only useful if semantic drift is controlled. Prior research emphasizes that migration quality depends heavily on scaffolding and verification, not just language choice [8, 1, 9].

3 Research Questions and Hypotheses

We propose the following questions:

- **RQ1**: Can we generate a Rust implementation that preserves rmatch semantics for compatible regex subsets?
- **RQ2**: Which scaffolding artifacts most reduce manual porting effort while keeping defect rate low?
- **RQ3**: Can generated Rust code match or exceed Java rmatch performance in heavy-load regimes?
- **RQ4**: Can translation-validation techniques provide fast reject/accept feedback during iterative generation?

Working hypotheses:

- **H1**: A contract-first scaffold (API + trace schema + oracle harness) reduces semantic regressions versus direct code translation.
- **H2**: Differential testing against stable corpora/pattern bundles catches most migration defects before expensive benchmarks.
- **H3**: Rust performance gains, if any, will emerge first in memory-pressure regimes, not necessarily at tiny workloads.

4 What Scaffolding Codex Needs

This section is the core answer to "what sort of scaffolding is needed from the data that is here?"

4.1 Scaffolding contract

Scaffold	Purpose	Minimum concrete artifact
S1: API contract freeze	Prevent interface drift during generation	Versioned matcher API spec (method signatures, threading expectations, action callback semantics)
S2: Semantic oracle corpus	Define correctness targets independent of implementation language	Fixed pattern/corpus bundles with hashes and expected outputs (counts + checksums)
S3: Regex-feature compatibility table	Avoid invalid equivalence claims across engines	Machine-readable matrix of supported constructs and known semantic deltas
S4: Execution trace schema	Enable translation validation at internal-state granularity	Optional NDFA/DFA transition trace format (JSONL/protobuf) from Java baseline and Rust candidate
S5: IR snapshot export	Decouple front-end parse from backend execution	Canonical serialized automata/intermediate representation for cross-language replay
S6: Differential harness	Fast reject for semantic mismatches	Runner that executes Java and Rust on identical workloads and diffs outputs deterministically
S7: Failure taxonomy	Preserve evidence and accelerate triage	Standard status classes: ok, timeout, stall, oom, semantic-mismatch, compile-failure
S8: Performance telemetry schema	Comparable benchmarking across campaigns	Standard fields for compile time, scan time, allocations, RSS peaks, CPU time, machine metadata
S9: Safety boundary policy	Keep unsafe Rust auditable	Explicit list of modules allowed to use `unsafe`, with justification and tests
S10: Generation ledger	Make LLM-driven changes reproducible	Per-module prompt + patch + test outcome log tied to commit SHA

4.2 Why these scaffolds are necessary

S1–S3 establish a stable contract and avoid ambiguous "compatibility" definitions. S4–S6 support translation validation and differential correctness [13, 4, 3]. S7–S8 enable reproducible analysis and publication-grade reporting [5, 7]. S9–S10 address Rust-specific trust and audit concerns observed in migration literature [6, 12, 11].

5 Method: Codex-Assisted Migration Protocol

We propose a phased method.

5.1 Phase A: Baseline freeze and observability

- Freeze canonical Java baselines and dataset IDs.
- Ensure deterministic result exports (counts + checksums).
- Add optional internal trace emission for NDFA/DFA transitions.

5.2 Phase B: Rust compatibility shell

- Create Rust crate with API mirror of Java matcher surface.
- Add CLI adapter so existing benchmark framework can invoke Rust candidate exactly like existing engines.
- Implement no-op/stub internals that compile and run smoke tests.

5.3 Phase C: Module-by-module generation

Generate and validate in dependency order:

1. regex parser + compiler data structures
2. NDFA node graph + epsilon-closure logic
3. DFA caching and transition mechanics
4. match-set runtime and action callbacks
5. partitioned multi-matcher scheduling

Each step is accepted only if differential tests pass against Java oracle for the targeted feature subset.

5.4 Phase D: Translation validation loop

For every generated module:

- run property-based equivalence checks (Java vs Rust outputs) [4]
- run corpus-level differential tests
- run bounded symbolic/fuzz campaigns on parser and state transitions [3]

Reject modules with unexplained semantic drift; keep all artifacts for postmortem.

5.5 Phase E: Performance characterization

Integrate Rust engine into the existing containerized/GCP workflow and run comparable cohorts only (same machine type, architecture, dataset URI, image policy), following the project fairness protocol and rigorous benchmarking guidance [5, 7].

6 Experimental Design

6.1 Dependent variables

- **Correctness**: match-count equality, checksum equality, and mismatch rate.
- **Performance**: mean total runtime, compile/scan split, variance, timeout/stall rate.
- **Engineering cost**: automated patch acceptance rate, manual edits per module, defect discovery latency.

6.2 Acceptance criteria

Define:

$$\text{CompatibilityScore} = 1 - \frac{\text{mismatching scenarios}}{\text{scenarios tested}}$$

$$\text{PerformanceRatio}_{\text{Rust/Java}} = \frac{T_{\text{Rust}}}{T_{\text{Java-rmatch}}}$$

Recommended stage gates:

- Stage 1: CompatibilityScore ≥ 0.999 on canonical non-RE2J-equivalence subset.
- Stage 2: No unexplained major mismatches on heavy-load scenarios.
- Stage 3: Median $\text{PerformanceRatio}_{\text{Rust/Java}} \leq 1.05$ on targeted production cohorts.

7 Threats to Validity

- **Semantic confounding**: regex dialect differences can masquerade as migration bugs.
- **Benchmark confounding**: architecture or memory pressure differences can distort comparisons.
- **Tooling confounding**: generated code quality may vary by prompt/template evolution.
- **Observability gaps**: missing memory or trace detail can hide root causes.

Mitigation is scaffold-driven: stricter cohorting, stronger trace/checksum evidence, and explicit validity checklists.

8 Concrete Deliverables

To make Codex effective, the repository should add:

1. `rust-port/api_contract.md` and `rust-port/compat_matrix.json`
2. `rust-port/oracle/` with hashed datasets and expected outputs
3. `rust-port/trace_schema.json` and Java trace emitter
4. `engines/rmatch-rust/` benchmark adapter wired into existing framework
5. `rust-port/generation_ledger/` for reproducible Codex iterations

9 Conclusion

The migration is feasible if treated as a scientific engineering program rather than raw transpilation. The key is scaffolding: explicit contracts, reproducible oracles, traceable generation, and rigorous cross-language validation. With this in place, the current Java implementation can serve as high-value scaffolding for a Rust-compatible engine while preserving comparability to existing benchmark campaigns.

References

[1] Omar Agha, Jonathan Schanck, Saikrishna Patnaik, Karthikeyan Bhargavan, et al. Rustmap: From legacy c to verified rust through semantic program mapping, 2025.

[2] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.

[3] Cristian Cadar, Daniel Dunbar, and Dawson Engler. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.

[4] Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming*, pages 268–279, 2000.

[5] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[6] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: Securing the foundations of the rust programming language. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 2, pages 66:1–66:34, 2018.

[7] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pages 63–74, 2013.

[8] Felix Klock, Meng Ren, Reynir McCallum, Jatin Pattnaik, Hoang-Hai Nguyen, Gabriel Scherer, Emina Torlak Lambert, et al. Automating the translation of c to rust. `https://cacm.acm.org/research-highlights/automating-the-translation-of-c-to-rust/`, 2025. Communications of the ACM research highlight; accessed 2026-03-03.

[9] Tian Lei et al. C2rustbench: Benchmarking c to rust translation, 2025.

[10] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[11] Pranav Nagappa, Rishi Goyal, et al. C2saferrust: End-to-end automated translation from c to safer rust, 2025.

[12] Hoang-Hai Nguyen, Yuan Huang, and Martin Vechev. In rust we trust: A transpiler from unsafe c to safer rust. *Proceedings of the ACM on Object-Oriented Programming, Systems, Languages, and Applications*, 6(OOPSLA2):1–26, 2022.

[13] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *LNCS*, pages 151–166, 1998.

[14] Ken Thompson. Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422, 1968.

[15] John Toman, Anders Møller Kragh-Hansen, et al. Safetrans: Towards sound and efficient automated c-to-rust translation, 2025.

[16] Xiaoqing Wang, Yang Hong, Harry Chang, Ihn Park, Geoff Langdale, Jiayu Hu, and Heqing Zhu. Hyperscan: A fast multi-pattern regex matcher for modern cpus. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 631–648, 2019.